
IMAGE TRANSLATION USING CONDITIONAL GENERATIVE ADVERSARIAL NETWORKS

CS:408 ADVANCED MACHINE LEARNING

Aditya Parameswaran
Ashoka Scholars Program '19
Ashoka University
aditya_asp19@ashoka.edu.in

Dibyendu Mishra
Ashoka Scholars Program '19
Ashoka University
dibyendu.mishra_asp19@ashoka.edu.in

Sai Vineet Reddy Thatiparthi
Undergraduate '19
Ashoka University
saivineetreddy.thatiparthi_ug19@ashoka.edu.in

Instructor: Dr.Ravi Kothari

May 3, 2019

ABSTRACT

Problems in image processing, graphics and vision require translating an image from one domain to another. One might want to reconstruct a damaged image, increase resolution of an image, or adapt an image to the visual style of another image. While these problems belong to the same domain, they often require different algorithms to be solved. In this project, we explore the use of conditional adversarial networks towards realising a generalised approach towards image translation problems. We demonstrate that these networks are effective at changing day light images to ones having a night setting, getting satellite maps corresponding to google maps, converting stereo annotated images of cityscapes to the original images and inpainting damaged pictures, all through the same network.

1 Introduction

Most of the work with respect to translation has primarily been done in the field of language translation which involves text, speech etc. Image translation can be seen as an extension to this, only with image data instead. In automatic image translation, we express a scene belonging to one domain that we have access to into an image belonging to another domain, which in many cases might be hard to access. This, of course, is made possible only when enough training data is available. Our model aims to prove that, regardless of the domain, the same neural network can translate images belong to different domains at once. These networks not only learn the mapping from input image to output image, but also learn a loss function to train this mapping.

For the above task we look at GANs, which learns a loss function on its own to say whether an image is fake or a real image. Simultaneously, the generator part of the network tries to flush out realistic looking images in the hope of fooling the discriminator part of the network. The advantage with this method over CNNs is that blurry images will not be tolerated because it would be classified as fake. A CNN would simply try to reduce Euclidean distance which is minimized by averaging all plausible outputs. But a simple GAN alone cannot solve the problem at hand because we don't want it output anything from the latent space but rather a specific type of output using features learnt during training.

We achieve this by using Conditional Generative Adversarial Networks. We use Conditional GANs as they allow us to put a constraint on the possible GAN outputs such that we get the specific translation we are looking for.

2 Related Work

In an unconditioned generative model, there is no control on modes of the data being generated. However, by conditioning the model on additional information it is possible to direct the data generation process. Mirza et al in their paper, introduce the conditional version of generative adversarial nets, which can be constructed by simply feeding the data, y that one wishes to condition on to both the generator and discriminator.

The image-conditional models have tackled image prediction from a normal map, future frame prediction, product photo generation, and image generation from sparse annotations (for an autoregressive approach to the same problem). Several other papers have also used GANs for image-to-image mappings, but only applied the GAN unconditionally, relying on other terms (such as L2 regression) to force the output to be conditioned on the input.

Our project is inspired by Isola et. al.'s paper on image to image translation problems using conditional GANs. They use a "U-Net"-based architecture for their generator, and for their discriminator they use a convolutional "PatchGAN" classifier, which only penalizes structure at the scale of image patches. We try to produce some results from their paper and later on frame our own problem in the image inpainting domain.

3 Method

3.1 Model Details

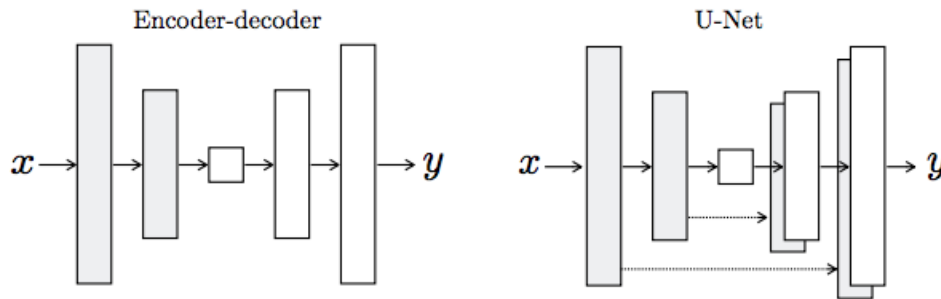
The GAN is converted to a CGAN using a simple tweak. GANs usually have a mapping from random noise vector z to output image y , $G : z \rightarrow y$. We want to condition the output on some input x which we would like the generator to imitate. Therefore the CGAN finds a mapping from random noise vector z, x to output image y , $G : \{x, z\} \rightarrow y$. To do this we create inputs in form of image pairs (real image, conditioned image) and feed it to the discriminator, the generator creates image pairs in the form (fake image, real image) feeds it to the discriminator. The generator therefore tries to create realistic (fake conditioned image, real image) pairs that fool the discriminator. The objective of an unconditioned GAN looks like this:

$$L(G, D) = E[\log(D(y))] + E[\log(1 - D(G(x, z)))]$$

For the conditional GAN the equation would change to:

$$L(G, D) = E[\log(D(x, y))] + E[\log(1 - D(x, G(x, z)))]$$

3.2 Generator



Generators in usual Generative Adversarial Networks take, as input, a vector of random noise. This noise is then upsampled, by performing deconvolutional operations, into an actual image that makes some sense. The generator being used in our model resembles an auto-encoder instead of a traditional generator. However, our generator also differs from a traditional auto-encoder as it employs skip connections; connecting the output of layers before encoding to layer after encoding. This allows the image being decoded to obtain data before the encoding, so that even if data is lost in the process of encoding, decoding can still recover this data.

Figure 1: U-Net Architecture

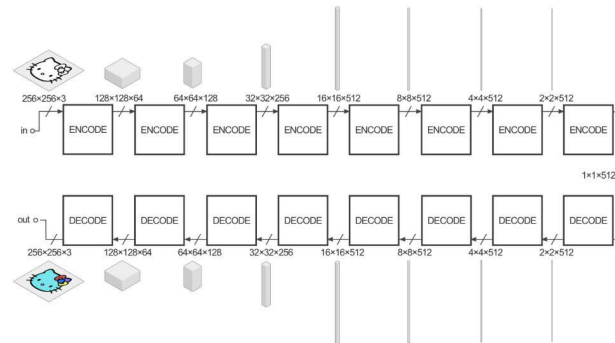
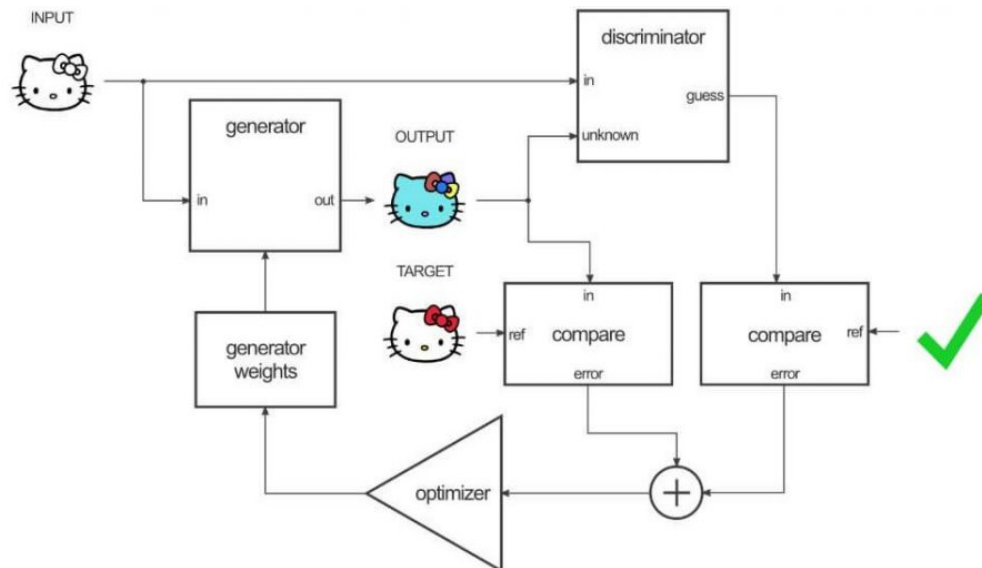


Figure 2: GAN architecture



Such an architecture is known as U-Net. These skip connections do not require any resizing or projections. This is achieved by ensuring that the spatial resolution of the layers being connected are the same.

3.3 Discriminator

The variant of the discriminator used in our model is known as a PatchGAN discriminator. The PatchGAN is a Markovian discriminator which works by classifying $N \times N$ square images, or patches, of the image as real or fake. This only penalizes structure at the scale of patches.. We run this discriminator convolutionally across the image, averaging all responses to provide the ultimate output of D. The following are outputs with different patch sizes:



4 Experiments

4.1 Datasets

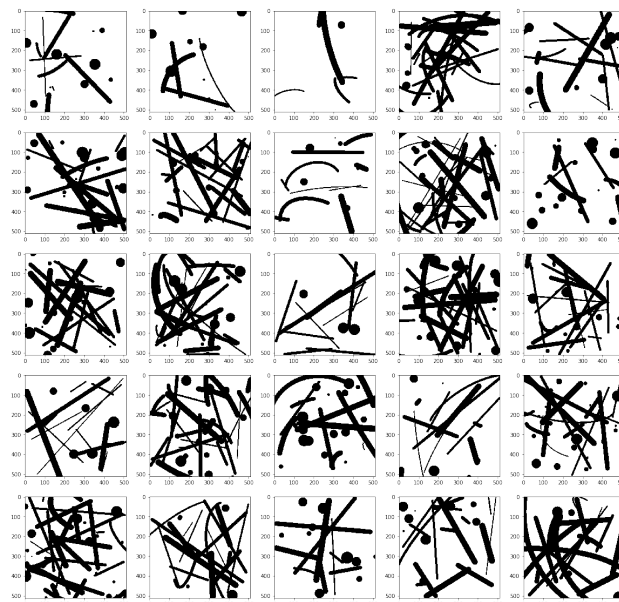
For our experiments and to explore the generality of our model we trained our network on four different datasets:

1. **Cityscapes** - Cityscapes is comprised of a large, diverse set of stereo video sequences recorded in streets from 50 different cities. 5000 of these images have high quality pixel-level annotations. We divided the data into 70:30 split, 70 percent of the data was used for training while 30 percent of the data was used for testing and validation. The dataset had annotated images of a street scene, these annotations included things such as light post, fence, zebra crossing, etc.
2. **Maps** - Maps aerial photograph 1096 training images scraped from Google Maps. We have access to their corresponding satellite images. The maps have two images, one depicting the roads and streets and the other a satellite image of the same scene.
3. **Day2Night** - There are totally 101 images captured. Each image is captured in various seasons, weathers, setting etc. Images are annotated by 40 attributes such as sunny, night, gloomy, winter etc. Each image is this represented by 40 features in a CSV file. We have only considered images have considered images which have 'Sunny' attribute > 0.8 and 'Night' attribute > 0.8 .
4. **Caltech-UCSD Birds** - CUB-200 dataset is an image dataset with 6,033 photos of birds spanning 200 species. We obfuscated the images using our masking method and created our inpainting dataset.

4.2 Pre-processing

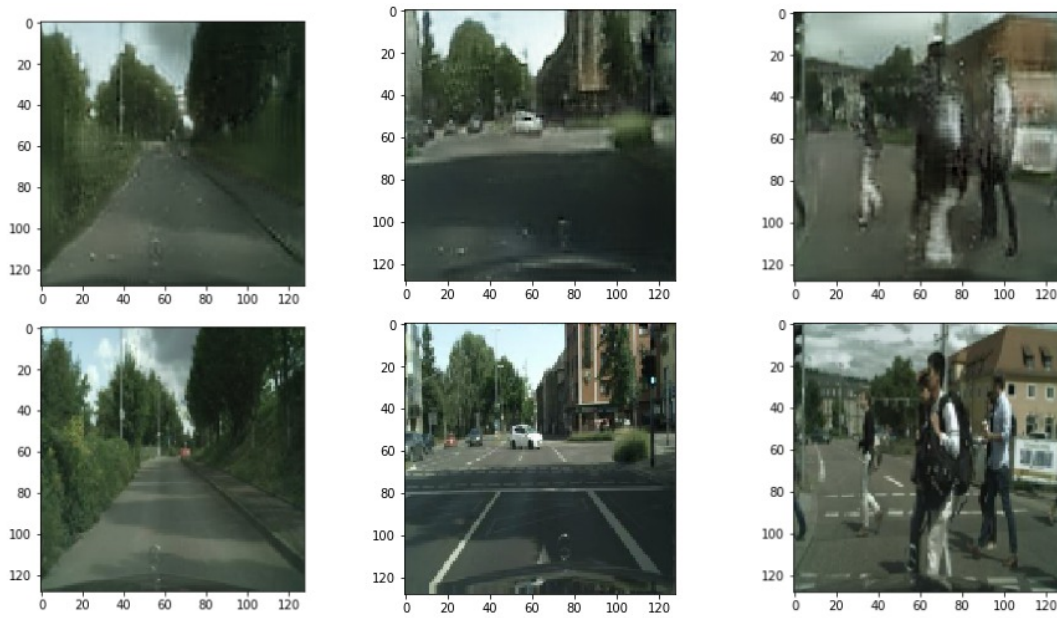
All the images that we have used, came in pairs as one image hosting two conjoined images. This meant that we had to first split the images in such a way that, one image became our input and the other would become our target. Since we had limited time, we also had to transform the images to a small uniform standard resolution. After testing with different input sizes and the memory and time requirement to train it, we decided that an image of the resolution 128x128 would work the best as it would allow us to see the results of the network that can be inspected with the naked eye and not take too much time to train.

For our UCSD Birds dataset, we created a simple mask generator function which uses OpenCV to draw some random irregular shapes like circles, lines, and ellipses of varying thickness which are then used for masking images from the birds' dataset. This would give us a dataset with obfuscated images and corresponding ground truth images. Here are a few examples of the masks we used:



4.3 Cityscapes

We trained our model for 200 epochs on 2975 images. The results of the generator after 200 epochs on the testing dataset are given below. The image on the first row are the generated images and the images on the bottom row are the actual ground truth.



One interesting that we noted here was how the reconstruction of people in a scene was better when then patch size of the GAN was large but this meant that the overall picture looks blurry. The picture below regarding the different patch sizes should provide more clarity on the matter.

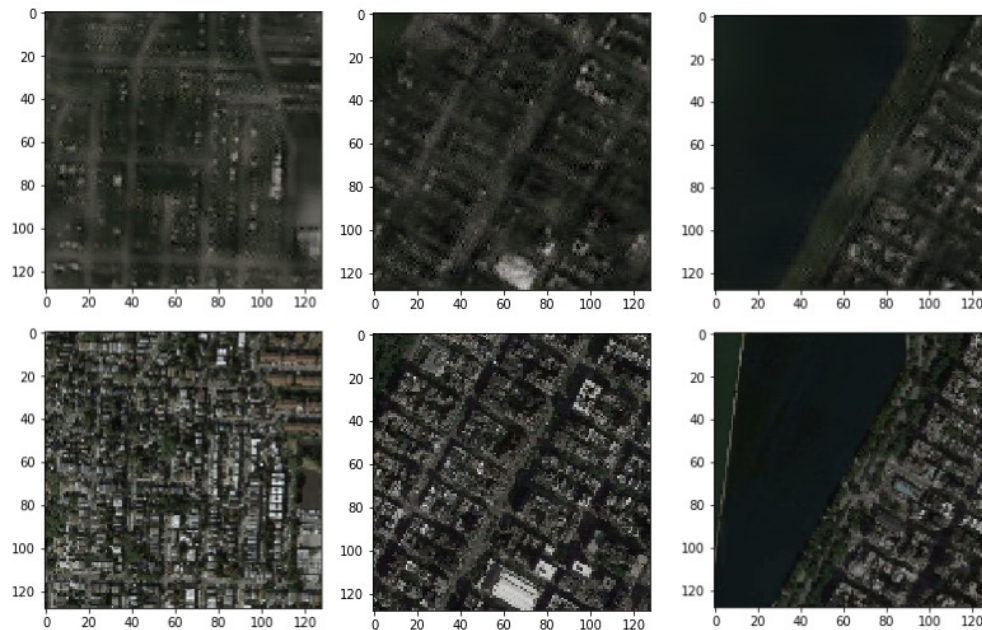
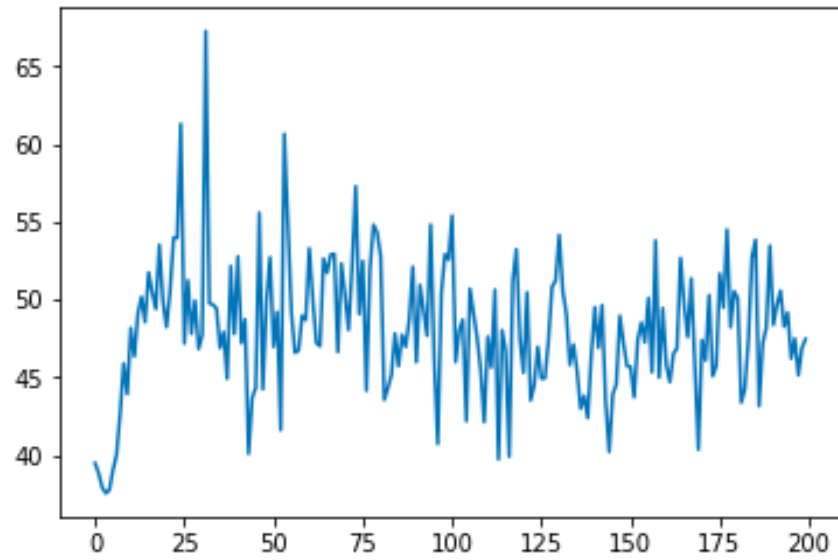


The loss, of both the discriminator and the generator, on plotting, looked like this. As one can see and expect, the modules of the GAN are fighting against each other; the increase of loss on one meant the decrease in loss of the other.

4.4 Maps

We trained our model for 200 epochs on 1950 images. The results of the generator after 200 epochs on the testing set were as follows. The model was given, as input, a Google Maps' map view of a square area. The expected output was the satellite image of the same area. The results are given below. The image on the first row are the generated images and the images on the bottom row are the actual ground truth.

Figure 3: Discriminator Loss for Cityscapes



The different patch sizes of the GAN did not play a large role in the reconstruction as you can see below.
Patch Sizes - 8×8 , 32×32 , 70×70

Figure 4: Generator Loss for Cityscapes

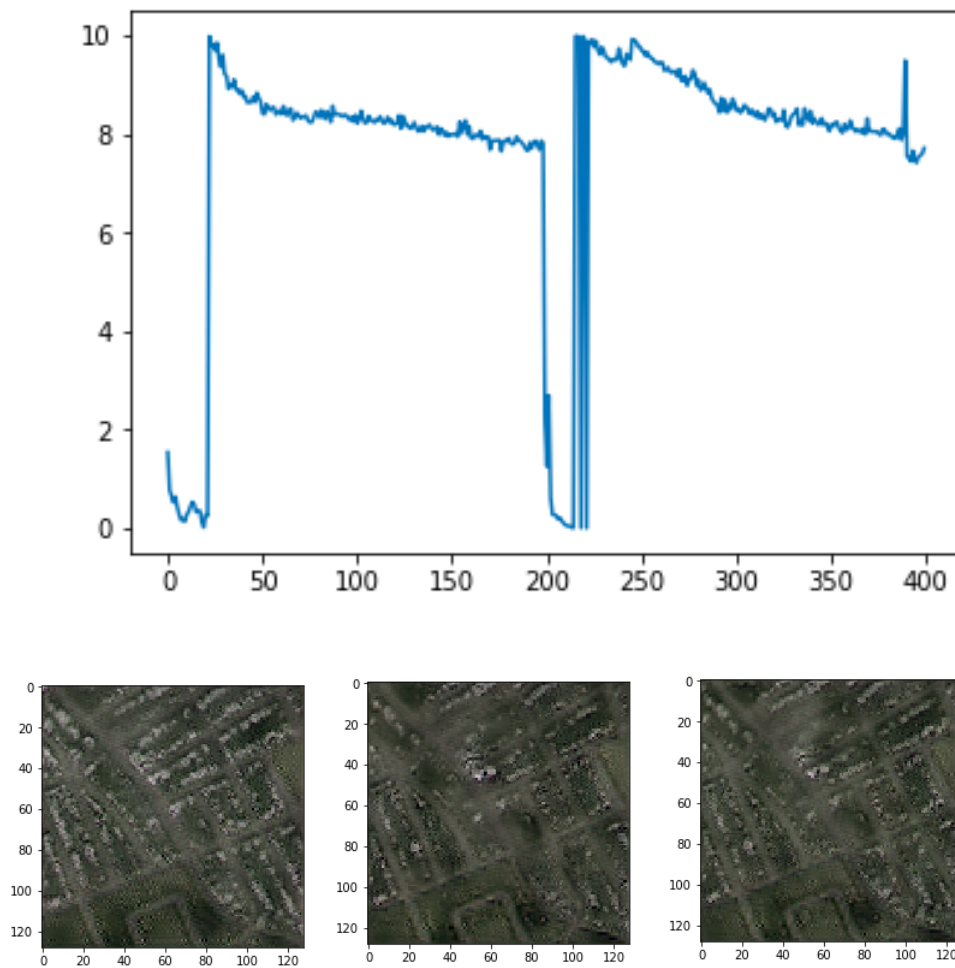
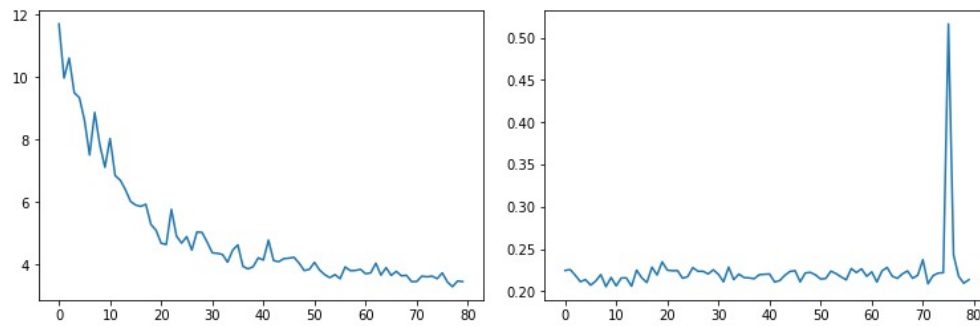
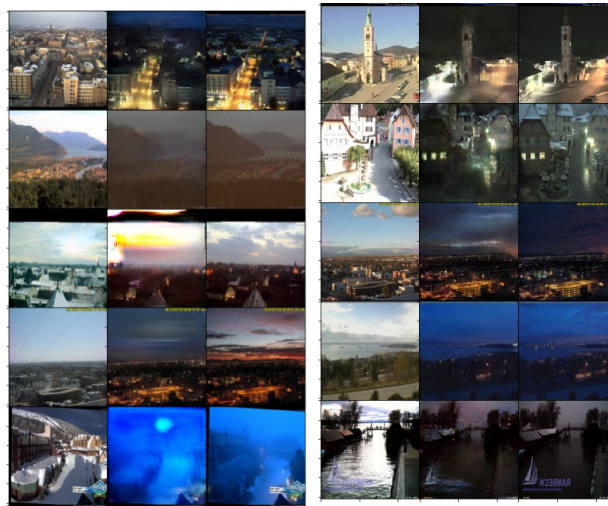


Figure 5: Generator Loss and Discriminator loss of



4.5 Day2Night

We gave as input, an image of scene taken in the day and the expected output (night image). The expected output, is another photo of the same scene taken at night. There were a total of 500 images that were trained for 200 epochs. The results of the generator are given below. The images in the first column are the input, the images on the middle column are the respective generated images while the images on the right most column are the ground truth.



The different patch sizes of the GAN did not play much of a role in the reconstruction. We did the run the model on an image of Ashoka University to see how it behaved when encountered with an image not part of the dataset.

4.6 Caltech-UCSD

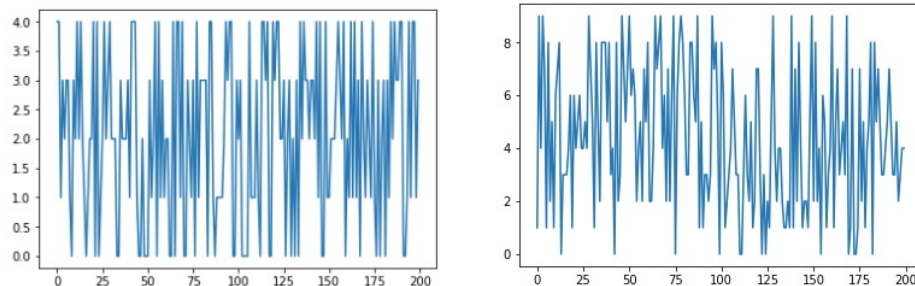
We gave as input, an obfuscated image of the birds using the masking technique and the expected output which was the original image. There were a total of 1000 images that we trained on for about 50 epochs.



The images in the first column are the input, the images on the middle column are the respective generated images while the images on the right most column are the ground truth.

While this problem could also have been tackled well by an autoencoder, our method suggests that the same network used for the earlier gave some really good results.

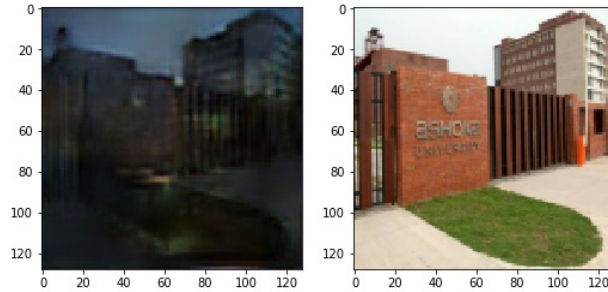
The losses of the generator and the discriminator are given below. The graph on the left is the loss of the discriminator and the graph on the right is the loss of the generator over 200 epochs.



5 Analysis

In general the results obtained for the different datasets were fairly good given the amount of training data available and the number of training epochs. But the model did perform better on some datasets as compared to others. This was specially seen in the obfuscated image dataset which seemed to achieve the reconstruction with a very low loss when compared to the *Cityscapes* dataset. As discussed, some reasons this could be true is because the object in focus was always a bird as opposed to the various objects encountered in the other datasets. For examples, the obfuscation dataset just had to encounter birds alone; the probability distribution of the dataset was easier to reproduce as there were fewer objects present in a given image of the dataset. Alternatively in the image obfuscation task the expected output was not a whole lot different than the obfuscated image, so it was not necessarily a translation between two completely different domains such as the aerial $\rightarrow$ map or the annotated street images $\rightarrow$ street scene, which might make the reconstruction task relatively easier.

While we dealt primarily with image to image translation, we also happened to do a style transfer in one of the datasets which pertained to the day to night translation. This is because unlike the other datasets we generate an image modifying the style of the input image while still preserving its content. When we tested it on a new image of the 'Ashoka University Gate'. It essentially performed a style transfer between existing night images produced by the generator and the new image to produce a night skyline for the Ashoka Image.



6 Conclusion

Using Conditional Generative Adversarial Networks to perform Image-to-Image translation yields great results considering the possibilities of such a model are endless. We are able to achieve satisfactory results on all the datasets we chose without ever having to change the model. While the training process is computationally intensive and time consuming, this only needs to be done once. One can save the weights of the corresponding to different datasets and perform operations on all of them instantaneously.

References

- [1] P. Welinder and S. Branson and T. Mita and C. Wah and F. Schroff and S. Belongie and P. Perona. Caltech-UCSD Birds 200.
- [2] Mehdi Mirza and Simon Osindero. Conditional Generative Adversarial Nets. In *arXiv preprint arXiv:1411.1784*, 2018.
- [3] Christopher hesse Image-to-Image Demo <https://affinelayer.com/pixsrv/>
- [4] Isola, Phillip and Zhu, Jun-Yan and Zhou, Tinghui and Efros, Alexei A. Image-to-Image Translation with Conditional Adversarial Networks. *arXiv preprint*, 2016.
- [5] Isola, Phillip and Zhu, Jun-Yan and Zhou, Tinghui and Efros, Alexei A. Image-to-Image Translation with Conditional Adversarial Networks. *arXiv preprint*, 2016.